

杨凌国合外贸综合服务平台 1.0 代码复查与建设建议

由 Markdown 阅读资料转换生成 · 通用自适应阅读版 · 适合转发和专家审阅

杨凌国合外贸综合服务平台 1.0 代码复查与建设建议

复查日期：2026-04-25

复查对象：/Users/wangbin/Documents/Codex/review_src2/autoWorkPlatform-v1.0.0

一、总体结论

这次已经复查到真实源码和部署包。平台 1.0 具备演示型 MVP 的雏形：有登录注册、租户、用户、配置、模板、合同、商品、订单、报关、进口台账、文件、供应商客户、退税展示等模块。但按“省级外贸综合服务平台”和“国有农产品外贸企业生产系统”的标准，目前还不适合直接上线生产。

核心判断：

- 当前最严重问题不是界面不够漂亮，而是数据库脚本与代码不一致、安全边界薄弱、部署配置不规范、业务闭环不足。
- 1.0 更像内部演示系统，离“可审计、可监管、可接入、可运营、可扩展”的省级平台还有明显差距。
- 不建议立刻用 Rust 全量重写。建议先用 2 到 8 周把现有 Java/Spring Boot 版本修到可用、可测、可部署，再选择部分高并发、高安全、高性能模块用 Rust 渐进式改造。
- 平台建设重点应从“做 CRUD 页面”转向“围绕外贸真实业务闭环建平台”：订单、合同、单证、报关、物流、收汇、结汇、退税、信保、融资、风控、审计、统计决策。

二、代码复查发现

PO-1 数据库脚本与代码严重不一致

这是当前第一优先级问题。sql/init.sql、deploy/sql/init.sql、backend/src/main/resources/db/init.sql 都只建到了 biz_partner，没有创建多张代码已经使用的表。

证据：

- ImportLedger 映射到 biz_import_ledger：
backend/src/main/java/com/trade/platform/module/ledger/entity/ImportLedger.java
- 但 SQL 初始化文件没有 biz_import_ledger、biz_import_ledger_goods、biz_import_ledger_file。
- CustomsReview 映射到 biz_customs_review，SQL 也没有该表。

- `CustomsDeclaration` 代码里有 `ledgerId`、`splitSeq`、`reviewStatus`、`submitTime`、`submitBy`、`reviewTime`、`reviewBy`、`deleted`：
`backend/src/main/java/com/trade/platform/module/customs/entity/CustomsDeclaration.java:102`
- SQL 里的 `biz_customs_declaration` 没有这些字段：`sql/init.sql:213`
- `Template` 代码里有 `filePath`、`fileName`、`fileSize`、`pdfPath`：
`backend/src/main/java/com/trade/platform/module/template/entity/Template.java:102`
- SQL 里的 `biz_template` 没有这些字段：`sql/init.sql:102`

影响：

- 进口台账、报关审核、模板上传编辑等功能很可能运行时报“表不存在”或“字段不存在”。
- 部署包内的 jar 也包含台账代码，但内置 SQL 仍然缺表，说明不是单纯源码遗漏，而是交付包整体不一致。

整改建议：

1. 立即补齐数据库迁移脚本，不要再只靠 `init.sql`。
2. 引入 Flyway 或 Liquibase，建立 `V1__init.sql`、`V2__ledger.sql`、`V3__customs_review.sql` 这种可追踪迁移。
3. 所有实体字段、Mapper、SQL、测试数据必须做一次全量对照。
4. 以后每次新增表字段必须配套迁移脚本和回滚说明。

PO-2 认证和权限存在生产风险

证据：

- 注册 DTO 要求前端传入角色：
`backend/src/main/java/com/trade/platform/module/user/dto/RegisterDTO.java:25`
- 注册服务直接保存前端提交的角色：
`backend/src/main/java/com/trade/platform/module/user/service/UserService.java:102`
- 审批通过时直接用申请表里的角色创建用户：
`backend/src/main/java/com/trade/platform/module/user/service/UserService.java:102`
- 权限拦截只判断角色字符串是否匹配注解：
`backend/src/main/java/com/trade/platform/security/PermissionInterceptor.java:102`
- 报关审核的 `approve/reject/release/revoke` 控制器没有角色注解：
`backend/src/main/java/com/trade/platform/module/customs/controller/CustomsController.java:102`

影响：

- 前端可以申请 `ADMIN` 或 `GUOHE` 等高权限角色，如果审批人员误点通过，就会产生越权账号。
- 报关审核、放行等业务动作没有区分制单人、审核人、国合员工、企业员工。

- 对省级平台来说，缺少“岗位、部门、租户、数据范围、审批流节点”的权限模型。

整改建议：

1. 注册接口只能申请普通企业用户，国合员工和管理员必须由后台创建或邀请。
2. 建立 RBAC 权限表：用户、角色、菜单、按钮、接口、数据范围。
3. 关键业务动作必须使用流程权限：制单、复核、审批、放行、退回、撤回不能由同一个岗位任意完成。
4. 增加操作审计表，记录操作者、IP、设备、前后状态、业务编号、时间。

PO-3 密码和 JWT 配置不安全

证据：

- JWT 过期时间计算为 `expiration * 1000`：
`backend/src/main/java/com/trade/platform/security/JwtUtil.java:22`
- 配置里 `jwt.expiration: 86400000`，看起来是毫秒，实际会被乘 1000，约等于 1000 天。
- 重置密码直接改成 123456：
`backend/src/main/java/com/trade/platform/module/user/service/UserService.java`
- 审批密码重置也改成 123456：
`backend/src/main/java/com/trade/platform/module/user/service/UserService.java`
- 默认管理员 `admin/admin123` 写入 SQL：`sql/init.sql:306`
- 开发配置里有 `root/root`、固定 JWT secret、内网 OnlyOffice 地址：
`backend/src/main/resources/application.yml:11`

影响：

- Token 过期过长，账号泄露后风险窗口非常大。
- 默认密码、固定密钥、数据库 root 用户都不符合国有企业生产系统安全要求。
- 测试账号和运维日志打进部署包，容易造成信息泄露。

整改建议：

1. JWT 过期时间统一按毫秒或秒，建议 access token 2 小时、refresh token 7 到 14 天。
2. 重置密码改为一次性随机临时密码或短信/邮件重置链接，首次登录强制改密。
3. 上线前删除所有默认账号，管理员首次初始化由部署脚本生成随机密码。
4. 密钥、数据库密码、OnlyOffice 密钥全部进入环境变量或密钥管理系统，不进源码、不进 jar。

PO-4 OnlyOffice 回调存在 SSRF 和未授权文件替换风险

证据：

- OnlyOffice callback 被排除 JWT：
`backend/src/main/java/com/trade/platform/config/WebMvcConfig.java:31`

- callback 说明“不需要 JWT 认证”：
`backend/src/main/java/com/trade/platform/module/template/controller/OnlyOfficeController.java:41`
- 回调体里的任意 url 会被后端直接下载并覆盖模板文件：
`backend/src/main/java/com/trade/platform/module/template/controller/OnlyOfficeController.java:41`
- 下载使用 `new URL(downloadUrl).openConnection()`：
`backend/src/main/java/com/trade/platform/module/template/controller/OnlyOfficeController.java:41`
- OnlyOffice JWT secret 为空：`backend/src/main/resources/application.yml:41`

影响：

- 外部请求可伪造回调，让服务器访问内网地址，形成 SSRF。
- 可诱导后端下载恶意文件，替换模板文件。
- 对涉及合同、报关、发票、装箱单的系统，这是严重安全风险。

整改建议：

1. 启用 OnlyOffice JWT，并校验回调签名。
2. callback URL 增加一次性 token、文档 key、来源 IP 白名单。
3. 后端下载 URL 必须限制域名或私有网段禁止访问，禁止访问 `localhost`、内网、云元数据地址。
4. 文件替换前校验文档类型、大小、hash，并保留历史版本。

P1-1 文件上传和公开访问不符合生产要求

证据：

- 上传只限制 5MB，没有通用文件类型白名单：
`backend/src/main/java/com/trade/platform/module/file/service/FileService.java:41`
- `originalFilename` 没有空值和无后缀处理：
`backend/src/main/java/com/trade/platform/module/file/service/FileService.java:41`
- `/uploads/**` 直接映射为静态资源：
`backend/src/main/java/com/trade/platform/config/WebMvcConfig.java:52`
- 删除文件只删数据库，不删物理文件：
`backend/src/main/java/com/trade/platform/module/file/service/FileService.java:41`

影响：

- 企业合同、证照、报关单、发票等敏感文件可能被 URL 猜测访问。
- 恶意文件上传后可能带来安全和合规问题。
- 删除记录后文件仍残留，形成数据泄露和合规风险。

整改建议：

1. 文件下载必须走鉴权接口，不直接暴露目录。

2. 建立文件类型白名单、MIME 校验、病毒扫描、大小分级限制。
3. 文件存储迁移到 MinIO/S3 兼容对象存储，使用临时签名 URL。
4. 删除采用软删除加物理归档策略，按档案制度保留和销毁。

P1-2 CORS 和异常处理不适合生产

证据：

- CORS 允许任意来源且允许凭据：
`backend/src/main/java/com/trade/platform/config/CorsConfig.java:13`
- 全局异常把内部异常消息返回给前端：
`backend/src/main/java/com/trade/platform/common/GlobalExceptionHandler.java:1`

影响：

- 跨域策略过宽，未来若改为 Cookie 或接入第三方系统，会扩大攻击面。
- SQL 错误、路径、内部配置可能暴露给用户。

整改建议：

1. 生产环境只允许企业正式域名、政务网域名、内网管理域名。
2. 前端只显示统一错误码和友好提示，详细堆栈只写安全日志。
3. 增加 `traceld`，便于运维排障。

P1-3 前端路由声明角色但没有真正拦截

证据：

- 路由里声明了 `roles`：`frontend/src/router/index.js:40`
- 路由守卫只检查 `token`，没有检查 `roles`：`frontend/src/router/index.js:212`
- `Token` 存在 `localStorage`：`frontend/src/utils/auth.js:5`

影响：

- 用户可以直接打开隐藏菜单路由，虽然后端部分会挡住，但前端体验和安全模型不完整。
- `localStorage` 中的 `token` 更容易受 XSS 影响。

整改建议：

1. 路由守卫补充角色检查。
2. 接口权限仍以后端为准。
3. 做 CSP、防 XSS、输入输出编码。
4. 中长期可改为 `httpOnly Secure Cookie` 加 CSRF 防护，或短 `token` + `refresh token` 机制。

P1-4 部署和交付包混杂

发现：

- 压缩包里包含 `deploy/trade-platform-1.0.0.jar`、`deploy/dist`、`deploy/logs/app.log`、测试产物、前端构建产物、IDE 文件、上传文件、`node_modules`、`backend target` 等。
- 交付目录 73MB，其中 `deploy` 68MB。
- `deploy/logs/app.log` 暴露了数据库连接失败信息：`deploy/logs/app.log:29`
- README 写 Java 17，但 `pom.xml` 是 Java 8：`backend/pom.xml:21`
- 本机没有 Java/Maven，无法完成后端编译；前端因 `node_modules` 未提取，`npm run build` 失败。

整改建议：

1. 建立标准仓库结构，只提交源码、脚本、文档，不提交日志、`target`、`dist`、`node_modules`、`uploads`。
2. 增加根目录 `.gitignore`。
3. 增加 `Dockerfile`、`docker-compose`、生产部署脚本。
4. 建立 CI：后端编译、单元测试、前端构建、依赖漏洞扫描、镜像构建。
5. 统一 Java 版本，建议新项目采用 Java 17 或 21 LTS。

P2-1 业务能力仍停留在基础管理

当前有：

- 租户、用户、配置、模板
- 合同、商品、订单
- 报关、进口台账、文件、供应商客户
- 退税页面

主要不足：

- 退税模块只是静态介绍页，没有业务数据和流程：`frontend/src/views/tax/index.vue:1`
- 缺少出口退税申报、单证归集、发票认证、退税进度、异常处理。
- 缺少收汇、结汇、付汇、核销、汇率风险管理。
- 缺少物流轨迹、冷链温控、仓储、集装箱、口岸节点。
- 缺少信保、买方资信、国家风险、应收账款预警。
- 缺少融资、授信、保理、订单融资、退税贷。
- 缺少农产品特有功能：溯源、检验检疫、农残检测、产地证、冷链温控、批次管理。
- 缺少省级监管驾驶舱：企业分布、商品结构、目的国、贸易额、风险预警、政策兑现。

三、对标国内先进平台的方向

建议对标时不要只看页面，而要看“服务链条”和“数据闭环”。

1. 对标国际贸易单一窗口

中国国际贸易“单一窗口”覆盖货物申报、舱单、许可证件、出口退税、金融服务、企业资质、原产地证、税费办理、口岸物流、公共查询等多类服务。参考页面：

- 中国国际贸易单一窗口应用列表：<https://nbapp.singlewindow.cn/>
- 云南单一窗口介绍提到国家标准版包括 14 大基本服务功能：<https://www.csa-expo.com/special/145>

对你们平台的启发：

- 国合平台不要重复建设海关政务系统，而要做企业侧业务协同和数据归集。
- 应把合同、订单、单证、报关、退税、物流、收结汇的数据打通，形成企业外贸档案。
- 与单一窗口、电子口岸、海关、税务、外汇、商务部门系统通过接口或导入导出衔接。

2. 对标阿里一达通

阿里一达通官方介绍的核心服务是通关、外汇、退税，并配套物流和金融。参考页面：

- 服务介绍：<https://onetouch.alibaba.com/activities/onetouch-services.html>
- 合作流程：<https://onetouch.alibaba.com/activities/onetouch-procedure.html>

对你们平台的启发：

- 平台价值不在“登记合同”，而在“企业把资料交上来后，平台能帮它完成外贸全流程”。
- 要设计服务工单：客户经理、单证员、报关员、财务、风控、审核人共同处理。
- 要沉淀贸易信用数据，服务后续融资、授信、风险定价。

3. 对标 XTransfer 等跨境金融平台

XTransfer 公开介绍其能力包括全球收款、本地收款、付款、结汇、换汇、多币种账户、风控服务。参考页面：

- XTransfer 官网：<https://www.xtransfer.cn/>
- 关于我们：<https://www.xtransfer.cn/about>

对你们平台的启发：

- 外贸服务平台必须重视资金流和风控，不只是单证流。
- 需要多币种、到账提醒、汇率、锁汇、收款合规、付款用途、反洗钱风控等能力。
- 国有企业平台可与银行、支付机构合作，不一定自建支付牌照能力。

4. 对标中国信保

中国政府网发布的政策文件明确出口信用保险在防风险、补损失、促融资、拓市场等方面的作用。参考页面：

- 中国政府网关于出口信用保险支持贸易高质量发展：
https://www.gov.cn/lianbo/bumen/202405/content_6952952.htm
- 中国出口信用保险公司监督管理办法：
https://www.gov.cn/zhengce/zhengceku/202412/content_6993969.htm

对你们平台的启发：

- 平台要把买方资信、国家风险、逾期应收、信保投保、理赔材料纳入流程。
- 对农产品出口尤其要关注买方拒收、质量争议、目的国政策变化、物流延误等风险。

四、建议建设的软件功能蓝图

1. 基础平台

- 组织与租户：国合、企业、合作社、报关行、货代、银行、保险、监管单位。
- 用户与权限：角色、岗位、数据范围、菜单、按钮、接口、审批权限。
- 流程引擎：申请、受理、补正、复核、审批、退回、办结、归档。
- 消息中心：站内信、短信、邮件、微信/企业微信。
- 审计中心：登录、导出、下载、审批、删除、配置变更全部留痕。
- 数据字典：国家地区、币种、港口、运输方式、贸易方式、HS 编码、监管条件。

2. 外贸业务闭环

- 客户与供应商：资质、联系人、信用等级、历史交易、风险标签。
- 商品与农产品档案：HS 编码、产地、批次、检验检疫、农残检测、包装规格、温控要求。
- 合同管理：合同评审、价格条款、付款条款、履约节点、到期预警。
- 订单管理：采购、销售、交付、装运、收款、开票、利润测算。
- 单证中心：发票、装箱单、合同、提单、产地证、植检证、报关草单、报关单、退税资料。
- 报关协同：台账、制单、复核、申报状态、回执、异常处理。
- 物流与仓储：订舱、集港、装柜、海运/铁路/空运轨迹、冷链温度、到港签收。
- 收汇结汇：应收账款、到账认领、结汇、汇率损益、逾期预警。
- 退税管理：发票采集、报关单匹配、退税率、申报批次、退税进度、差异处理。
- 信保与风控：买方资信、国家风险、限额申请、保单、出运申报、报损理赔。
- 融资服务：订单融资、退税贷、应收账款质押、信保保单融资。

3. 省级监管与运营

- 企业画像：注册地、行业、商品、目的国、贸易规模、活跃度。
- 外贸驾驶舱：进出口额、订单数、报关量、退税额、融资额、风险事件。
- 农产品专题：品类、产地、出口国家、价格指数、冷链异常、质量追溯。
- 政策匹配：补贴、展会、信保、融资、外综服政策自动匹配。
- 服务绩效：国合内部员工处理时效、工单积压、客户满意度。
- 风险预警：买方逾期、异常高频导出、报关退单、退税异常、敏感国家交易。

五、Rust 重构判断

是否建议全量 Rust 重写

不建议现在全量重写。

原因：

1. 当前最大问题是业务模型、数据库、权限、安全和部署，不是 Java 性能瓶颈。
2. 团队基础较弱，全量 Rust 会提高招聘、培训、维护成本。
3. Rust 生态很强，但企业管理系统的后台 CRUD、权限、流程、报表、Excel、Office 集成，用 Java/Spring Boot 或 Go 更容易快速交付。
4. 全量重写会让 1.0 的业务问题重新走一遍，风险大、周期长。

Rust 适合用在哪些地方

建议采用“Java 主平台 + Rust 高价值模块”的路线。

适合 Rust 的模块：

- 高性能文件解析：大批量 Excel、CSV、报关单、物流轨迹解析。
- 批处理任务：海量数据清洗、风控特征计算、报表预聚合。
- 安全网关：签名验签、加解密、接口限流、审计日志采集。
- 数据同步 Agent：与单一窗口、银行、物流、信保系统做稳定同步。
- 文档转换服务：单证生成、PDF 水印、文件 hash、病毒扫描前置。
- 风控规则引擎：多维度规则匹配和低延迟预警。

Rust 的好处：

- 性能高，适合大文件和高并发。
- 内存安全，减少空指针、越界、并发数据竞争等问题。
- 单文件部署方便，运行资源占用低。
- 对安全敏感服务更有优势。

Rust 的代价：

- 学习曲线陡，团队需要 3 到 6 个月适应。
- 生态在后台管理、低代码报表、Office 处理方面不如 Java 成熟。
- 开发效率前期可能下降。
- 需要建立跨语言接口规范、日志规范、部署规范。

推荐路线：

1. 近期 0 到 3 个月：修好 Java 版 1.0，不重写。
2. 中期 3 到 6 个月：把文件解析、数据同步、风控任务做成独立 Rust 服务。
3. 长期 6 到 18 个月：如果团队成熟，再考虑将网关、批处理、核心风控逐步 Rust 化。

六、推荐技术架构

当前修复版 1.1

- 前端：Vue 3、Vite、Element Plus。
- 后端：Java 17 或 21、Spring Boot 3.x、Spring Security、MyBatis-Plus 或 JPA。
- 数据库：MySQL 8 主从或 PostgreSQL 16。
- 缓存：Redis 7。
- 文件：MinIO。
- 搜索：OpenSearch 或 Elasticsearch，先不上也可以。
- 流程：Flowable、Camunda 8 或轻量自研审批流。
- 消息：RabbitMQ 或 RocketMQ，前期可用 Redis Stream。
- 部署：Docker Compose 起步，生产建议 Kubernetes 或至少容器化部署。

中长期 2.0

- API 网关：Nginx/OpenResty 或 Spring Cloud Gateway。
- 认证：统一身份认证、MFA、OAuth2/OIDC、CA/国密可选。
- 微服务拆分：用户权限、贸易业务、单证中心、文件中心、风控中心、数据中心。
- Rust 服务：文件解析、数据同步、风控计算、安全网关。
- 数据仓库：Doris/ClickHouse，用于驾驶舱和统计。
- 日志监控：Prometheus、Grafana、Loki/ELK、SkyWalking。

七、硬件配置建议

方案 A：试点版，50 到 200 家企业

适合内部试运行、县区或园区试点。

- 应用服务器 2 台：8 核 CPU、32GB 内存、300GB SSD。
- 数据库服务器 1 台：8 到 16 核 CPU、64GB 内存、1TB SSD，定期备份。
- 文件服务器或 MiniIO 1 台：8 核 CPU、32GB 内存、4TB 存储。
- 备份服务器 1 台：4 核 CPU、16GB 内存、4TB 到 8TB 存储。
- 带宽：公网 100Mbps 起，政务/内网按实际接入。

方案 B：省级生产版，1000 到 5000 家企业

- 负载均衡 2 台：Nginx/HAProxy，主备。
- 应用服务器 3 到 6 台：16 核 CPU、64GB 内存、500GB SSD。
- 数据库主库 1 台：32 核 CPU、128GB 内存、2TB NVMe SSD。
- 数据库从库 1 到 2 台：同主库或略低，用于读扩展和容灾。
- Redis 3 节点：8 核 CPU、32GB 内存。
- MiniIO 4 节点：每节点 8 到 16 核、32GB 内存、8TB 到 24TB 存储。
- 日志监控 2 台：16 核 CPU、64GB 内存、2TB 存储。
- 备份：本地每日、异地每周，关键数据至少保留 6 到 12 个月。
- 安全设备：WAF、防火墙、堡垒机、数据库审计、漏洞扫描。

方案 C：云上政企版

如果允许上云，建议使用政务云或国资认可云资源：

- SLB + ECS/容器服务。
- RDS MySQL/PostgreSQL 高可用版。
- Redis 高可用版。
- OSS/对象存储。
- WAF、云防火墙、日志服务、密钥管理 KMS。
- 异地备份和容灾。

八、部署环境建议

生产基础：

- 操作系统：Rocky Linux 9、Anolis OS、openEuler 或 Ubuntu Server LTS。
- JDK：Temurin/OpenJDK 17 或 21。
- Node.js：20 LTS 或 22 LTS，不建议生产使用过新的非 LTS。
- 数据库：MySQL 8.0 或 PostgreSQL 16。
- Redis：7.x。
- Nginx：1.24+。

- 容器：Docker + Compose 起步，后续 Kubernetes。
- 证书：HTTPS 全站启用，内部接口也建议 TLS。

安全基线：

- 禁止 root 账号直连数据库。
- 数据库、Redis、MinIO 不暴露公网。
- 所有密码进入环境变量或密钥管理。
- 管理后台启用 MFA。
- 上传文件走病毒扫描和权限下载。
- 开启数据库审计、登录审计、接口审计。
- 建立漏洞扫描和依赖升级制度。

九、推荐实施路线图

第 0 阶段：立即止血，1 到 2 周

- 补齐数据库表和字段，确保当前功能能跑通。
- 修复 JWT 过期时间、默认密码、重置密码、CORS、异常泄露。
- 关闭公开 `/uploads`，改为鉴权下载。
- 修复 OnlyOffice callback 安全校验。
- 删除交付包里的日志、target、dist、node_modules、uploads。
- 建立 `.gitignore`、README 部署说明、环境变量模板。

第 1 阶段：1.1 可用版，1 到 2 个月

- 引入 Flyway/Liquibase。
- 引入 Spring Security 或完善现有认证授权。
- 建立 RBAC 和审计日志。
- 把退税从静态页面改为真实业务模块。
- 完善单证中心、文件中心、业务流程状态机。
- 建立自动化测试和 CI。

第 2 阶段：1.5 业务闭环版，3 到 6 个月

- 合同、订单、单证、报关、物流、收汇、退税全流程打通。
- 建立服务工单和内部协同。
- 对接银行、物流、信保、单一窗口数据导入导出。
- 建立省级驾驶舱和农产品专题。

- 上线风险预警和企业画像。

第 3 阶段：2.0 平台化版，6 到 18 个月

- 多租户、多机构、多角色、多数据范围。
- 微服务或模块化单体演进。
- Rust 服务用于文件解析、风控、同步。
- 建立数据仓库和 BI。
- 通过等保测评、代码审计、渗透测试。

十、适合你们基础较弱团队的组织建议

建议角色配置：

- 产品经理 1 名：懂外贸流程，负责需求和验收。
- 后端工程师 2 到 3 名：Java/Spring Boot。
- 前端工程师 1 到 2 名：Vue。
- 测试工程师 1 名：接口测试、业务流程测试。
- 运维/DevOps 1 名：服务器、数据库、备份、安全。
- 外贸业务专家 2 到 3 名：国合内部人员兼职也可以，负责流程校验。
- 安全顾问阶段性参与：上线前做代码审计和渗透测试。

管理方法：

- 不要一次性追求“大而全”。
- 每两周一个版本，固定演示和验收。
- 业务需求必须写成流程图、状态图、字段表、权限表。
- 每个功能必须有测试用例和验收标准。
- 先把 10 条最常见业务流程跑顺，再做复杂功能。

十一、本次本地验证情况

已完成：

- 成功解压真实项目源码。
- 检查后端配置、pom、核心实体、Controller、Service、SQL、部署包。
- 检查前端路由、认证存储、退税页、仪表盘。
- 检查部署目录、日志、jar 内配置、测试脚本。

无法完成：

- 本机没有 Java Runtime 和 Maven，后端无法编译运行。

- 前端 `node_modules` 未完整提取，`npm run build` 报 `vite: command not found`。
- 因此本次结论以静态代码审查、部署包检查、日志检查为主。

建议下一步由开发商提供：

- 完整 Git 仓库。
- 数据库现网结构导出。
- 可运行测试环境。
- 最近一次部署包生成过程。
- 需求文档、接口文档、数据库设计文档。
- 管理员、国合员工、企业员工三类账号的测试数据。

本文档为内部研究和项目推进资料，请结合正式政策文件、合同文本和业务凭证审慎使用。